Ders 19 Çalışma Özeti

Ders 19 Çalışma Özeti

Genel Konular

- Dersin tarihi 9 Aralık 2020 Çarşamba, dönemin 10. haftası. Ders Zoom üzerinden yapıldı.
  - İlk 7 dakika dersin işleyişi hakkında soru-cevap ve duyurulardır.
  - Kaydın 00:57:40 ile 01:09:00 arası ara vermedir.
- Düzenleme uzaklığı (edit distance) ya da Levenshtein uzaklığı (Levenshtein distance)
  - Tanım ve kullanım alanı: yazım denetleyicileri (spell checker).
  - Dört işlem: kopyalama (copy), değiştirme (change, substitution), ekleme (insert), silme (delete).
  - Kaba kuvvet (brute force) ile hizalama denemeleri: MATHS ve ARTS.
  - Dinamik programlama tablosu, başlangıç değerleri ve bağıntı (recurrence relation).
  - İkinci örnek: HONDA ve HYUNDAI. Geriye doğru izleme ve birden fazla en iyi hizalama.
- En uzun ortak alt dizi (longest common subsequence, LCS)
  - Tanım: karakterlerin ardışık olması gerekmez, sıraları korunur.
  - Kullanım alanı: biyoinformatikte DNA dizilerinin benzerliği.
  - Örnekler: ATCTGAT ile TGCATA, BACDB ile BDCB.
  - Tablo, bağıntı ve "x'te atla" (skip x), "y'de atla" (skip y) fikri.
- Ödev ve proje duyuruları.

Hocanın Özellikle Vurguladığı Kısımlar

- Düzenleme uzaklığında kopyalamanın maliyeti yoktur. Değiştirme, ekleme ve silmenin maliyeti 1'dir.
- Tablonun bir satır ve bir sütun büyük kurulması gerekir. Boş dizgiye (boşluk) karşılık gelen ilk satır ve sütun, ekleme ve silme durumlarını temsil eder.
  - Bir kelimenin boş dizgiye uzaklığı harf sayısı kadardır. İlk satır ve ilk sütun $0, 1, 2, \ldots$ diye ilerler.
- Harfler aynıysa sol üst çaprazdaki değer olduğu gibi taşınır.
  - Harfler farklıysa çapraz, üst ve sol değerlerin en küçüğüne 1 eklenir.
- Optimal çözüme ulaşmak için alt problemlerin en iyi çözümlerini saklamak gerekir. Dinamik programlamanın özü budur.
- Uzaklığı bulmak yetmez. Hangi işlemlerin yapıldığı da tablodan geri izlenerek bulunur.
  - Aynı uzaklığı veren birden fazla hizalama olabilir.
- LCS'de karakterlerin arka arkaya gelmesi gerekmez. Aradaki fazla harfler atlanır.
- LCS'de eşit olmama durumunda hangi dizide atlandığını kaydetmek gerekir. Kodlarken $i + 1$ ya da $j + 1$ adımlarından biri seçilmelidir.

Kısa Tekrar Notları

- Düzenleme uzaklığı, bir dizgiyi diğerine çevirmek için gereken en az ekleme, silme ve değiştirme sayısıdır.
- Levenshtein bu fikri 1965'te ortaya attı.
- Başlangıç: $ED[i, 0] = i$, $ED[0, j] = j$.
- Bağıntı:
  - $x_i = y_j$ ise $ED[i, j] = ED[i - 1, j - 1]$.
  - Aksi hâlde $ED[i, j] = 1 + \min\{ED[i - 1, j - 1],\ ED[i - 1, j],\ ED[i, j - 1]\}$.

- $ED(\text{MATHS}, \text{ARTS}) = 3$ ve $ED(\text{HONDA}, \text{HYUNDAI}) = 3$.
- LCS bağıntısı:
  - $i = 0$ ya da $j = 0$ ise $C[i, j] = 0$.
  - $x_i = y_j$ ise $C[i, j] = C[i - 1, j - 1] + 1$.
  - Aksi hâlde $C[i, j] = \max\{C[i - 1, j],\ C[i, j - 1]\}$.
- $LCS(\text{BACDB}, \text{BDCB})$ uzunluğu 3: BCB ya da BDB.
- $LCS(\text{ATCTGAT}, \text{TGCATA})$ uzunluğu 4: TCTA, TGAT ya da TCAT.
- İki algoritmanın da zaman ve yer karmaşıklığı $\Theta(mn)$.

Detaylı Açıklamalar

Düzenleme uzaklığının tanımı ve kullanımı

- Levenshtein uzaklığı, iki dizi (sequence) arasındaki farkı ölçen bir dizgi ölçütüdür (string metric).
- SUNNY kelimesi örneğinde üç hata türü gösterildi:
  - Bir harf yanlış yazılır, örneğin N yerine M: değiştirme.
  - Bir harf eksik yazılır: silme.
  - Fazladan bir harf yazılır: ekleme.
- **Yazım denetleyici:** Önce kelimenin sözlükte olup olmadığına bakılır. Varsa sorun yoktur. Yoksa sözlükteki en yakın kelimeler önerilir.
  - Bir öğrenci, yazılan kelimeyi sözlükteki her kelimeyle karşılaştırmanın pahalı olacağını belirtti. Hoca bu soruyu sonraya bıraktı.

Kaba kuvvet ile MATHS ve ARTS

- İki kelime farklı biçimlerde hizalanabilir. Boşluk başa, sona ya da araya konabilir.
- Birinci hizalama: başa M eklenir, A eşleşir, T yerine R, H yerine T gelir, S eşleşir. Uzaklık 3.
- İkinci hizalama: M yerine A, A yerine R gelir, T eşleşir, H eklenir, S eşleşir. Uzaklık yine 3.
- Bütün hizalamaları denemek kaba kuvvet yaklaşımıdır. Dinamik programlama bu denemeyi bir tabloyla kısaltır.

Dinamik programlama tablosu

Satırlar MATHS, sütunlar ARTS. İlk satır ve sütun boş dizgiye karşılık gelir. Tablo Python ile doğrulandı:

	ε	A	R	T	S
ε	0	1	2	3	4
M	1	1	2	3	4
A	2	1	2	3	4
T	3	2	2	2	3
H	4	3	3	3	3
S	5	4	4	4	3

- M ile A farklıdır. Değiştirmenin maliyeti, sol üst çaprazdaki 0'a eklenir: $0 + 1 = 1$.
  - Derste bir öğrenci önce bu satırı $1, 2, 3, 4, 5$ diye doldurdu. Bir başka öğrenci, maliyetin çapraza eklenmesi gerektiğini fark etti. Doğrusu $1, 1, 2, 3, 4$.
- A ile A aynıdır. Çaprazdaki 1 taşınır.
- T satırında T ile R farklıdır. Üç komşunun en küçüğü 1, sonuç 2. Bir öğrenci önce yalnız üstteki hücreye bakıp 3 dedi. Doğrusu, üç komşunun en küçüğünü almaktır.
- Sonuç tablonun sağ alt köşesindedir: 3.

Dört durumun tablodaki karşılığı:

- **Kopyalama:** $x_i = y_j$. Değer sol üst çaprazdan maliyetsiz gelir: $ED[i - 1, j - 1]$.
- **Değiştirme:** Değer çaprazdan gelir, 1 eklenir: $ED[i - 1, j - 1] + 1$.

- **Ekleme / silme:** Değer üstten ($ED[i-1, j]+1$) ya da soldan ($ED[i, j-1]+1$) gelir. Hangisinin ekleme, hangisinin silme sayıldığı, dönüşümün hangi yönde düşünüldüğüne bağlıdır.

HONDA ve HYUNDAI

Tablo Python ile doğrulandı:

	ε	H	Y	U	N	D	A	I
ε	0	1	2	3	4	5	6	7
H	1	0	1	2	3	4	5	6
O	2	1	1	2	3	4	5	6
N	3	2	2	2	2	3	4	5
D	4	3	3	3	3	2	3	4
A	5	4	4	4	4	3	2	3

- Uzaklık 3. H, N, D, A eşleşir. O ile Y ya da U arasında bir değiştirme, bir ekleme vardır. Sonda I eklenir.
- **Geri izleme:** Sağ alt köşeden başlanır.
  - Harfler aynıysa ve çapraz değer eşitse çapraza gidilir.
  - Farklıysa, 1 eksik değeri veren komşuya gidilir.
  - İki komşu da uygunsa iki farklı en iyi yol vardır. Derste iki farklı hizalama gösterildi.

En uzun ortak alt dizi (LCS)

- Amaç: iki dizinin içinde, sırası korunarak ortak geçen en uzun alt diziyi bulmak. Karakterlerin bitişik olması gerekmez.
- DNA örneği: X = ATCTGAT, Y = TGCATA.
  - Öğrencilerden gelen cevaplar TCTA ve TGAT oldu. İkisi de uzunluğu 4 olan çözümlerdir.
  - Bir öğrencinin önerdiği CAT de ortak alt dizidir, ama uzunluğu 3 olduğu için en uzun değildir.
- Biçimsel tanım: $X = x_1 \dots x_m$, $Y = y_1 \dots y_n$. $S = s_1 \dots s_k$, her iki dizide de artan indislerle geçen dizidir.

LCS tablosu: BACDB ve BDCB

Satırlar BACDB, sütunlar BDCB. Tablo Python ile doğrulandı:

	ε	B	D	C	B
ε	0	0	0	0	0
B	0	1	1	1	1
A	0	1	1	1	1
C	0	1	1	2	2
D	0	1	2	2	2
B	0	1	2	2	3

- İlk satır ve sütun 0'dır, çünkü boş diziyle eşleşme yoktur.
- Karakterler eşleşince sol üst çaprazdaki değere 1 eklenir.
- Eşleşmeyince üstteki ve soldaki değerlerden büyük olan taşınır.
- İki komşu eşitse iki farklı yoldan gelinmiş olabilir. Bu, birden fazla LCS olduğunu gösterir.
- Sonuç 3. LCS'ler BCB ve BDB.

Ödev ve proje duyuruları

- Dersten sonra saat 15.00'te ödev oturumu yapılacak. Ödevin çözümü anlatılacak ve rastgele seçilen öğrencilerin ödevleri kontrol edilecek. Herkesin kontrolü yapılmayacak.

- Üçüncü ödev büyük ihtimalle Cumartesi verilecek. Konusu yine dinamik programlama olacak.
- Dönem projesi bir-iki hafta içinde açıklanacak.
- Geçen hafta saat 13.10'dan sonra derse girişi durdurma uygulaması bir kez yapıldı. Hoca bunun her hafta yapıldığı yönündeki söylentiyi düzeltti.
  - Bağlantısı kopup giremeyen öğrenci, bir arkadaşı aracılığıyla sohbetten hocaya yazarsa içeri alınır.
- Geçmiş konulardan örnek çözülmesini isteyenler, konuyu hocaya yazarsa uygulama saatinde örnek hazırlanacak.
- Gelecek hafta: LCS'nin sözde kodu ve geri izleme.

Ders sonrası notlar

- Bir öğrenci tablonun boyutunu "en uzun dizginin boyu çarpı kendisi" diye önerdi. Doğrusu $(m+1) \times (n+1)$ boyutlu bir tablodur. Hoca da sonra her iki tarafı birer büyüttü.
- Harfler aynıyken çapraz değeri doğrudan taşımak güvenlidir. Komşu hücreler arasındaki fark en fazla 1 olduğu için çapraz değer, diğer iki seçenekten (komşu + 1) hiçbir zaman büyük olmaz.
- Düzenleme uzaklığı ve LCS'nin ikisi de $\Theta(mn)$ zaman ve yer kullanır. Yalnız uzaklık isteniyorsa, iki satır tutularak yer $\Theta(\min(m,n))$'e indirilebilir. Ama o zaman geri izleme yapılamaz.
- ATCTGAT ve TGCATA için derste bulunan TCTA ve TGAT'a ek olarak TCAT de uzunluğu 4 olan bir LCS'dir.
- Yazım denetleyicide her kelimeyle karşılaştırma pahalıdır. Uygulamada aday kelimeler önce daraltılır. Örneğin uzaklığı 1-2 olan bütün düzenlemeler üretilip sözlükte aranır ya da BK-ağacı (BK-tree) gibi yapılar kullanılır.
- Levenshtein'ın makalesi 1965'te Rusça yayımlandı, İngilizcesi 1966'da çıktı.
- Altyazıda "Levenstein" diye geçen ad Levenshtein, "Delayed", "dilation" diye geçen terim delete (silme), "Long and Subsequence" diye geçen terim longest common subsequence'dir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.